1 **Design & Implementation of Human-Computer Interfaces**
NPTEL-MOOCS

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

2 **L24 – System Implementation Coding Basics**

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

3

4 **Next?**

- Writing code for implementation
- Code testing (different from prototype testing)

5

6 **Code Writing**

7 **Code Writing**

- Let's start with an example
- Consider a program to ADD two numbers (only the function)
  - Takes two integers as input
  - Produces the sum as output

8 **Code Writing**

9 **Code Writing**

10

Things to Remember

11 **Coding Standards/Guidelines**

- It helps to follow standard coding practices/guidelines
  - Helps readability and understandability
  - Good for teamwork

12 **Coding Standards/Guidelines**

- Organizations may have their own guidelines/standards
- There are some general rules we can follow

13 **Things to Remember**

- Indentation
  - One of the very basic thing
  - Improves readability

14 **Things to Remember**

- Global variables
  - Naming convention (should be different than others)
    - Eg, may always start with uppercase letters
    - Use prefix (GBL_Name)
    - Use suffix (Name_GBL)
    - …

15 **Things to Remember**

- Global variables
  - Number (limited is better)
  - Should be easy to find (may be single/limited number of definition files)

16 **Things to Remember**

- Module header (in each file) – typical things
  - Module name
  - Date on which module created
  - Author's name

17 **Things to Remember**

- Module header (in each file) – typical things
  - Modification history
  - Module summary

18 **Things to Remember**

- Module header (in each file) – typical things
  - Different functions supported, along with their input/output parameters
  - Global variables accessed/modified by the module
  - …

19 **Things to Remember**

- Naming conventions – maintain consistency
  - Global variable names may always start with capital letter
  - Local variable names may made up of small letters
  - Constant names may always be capital letters

20 **Things to Remember**

- Error and exception handling
  - Error conditions reporting should be standard
    - E.g., functions may return a 0 for error or 1 otherwise, consistently

21 **Things to Remember**

- Error and exception handling
  - Exception handling should be must

22 **Things to Remember**

- Code should be well-documented
  - A rule of thumb - at least one 'concise and explanatory' comment line on average for every 3-source lines
  - Good idea to create 'user manual' and 'technical manual'

23

Things to Avoid

24 **DO NOT's**

- Use coding style too clever or too difficult to understand
  - Many coders actually take pride in writing cryptic and incomprehensible code (e.g., writing entire code in one line)

```
for(int i=p; i<fin.size(); i+=(i&(i^(i-1))){fin[i]+=x;}
```

25 **DO NOT's**

- Use coding style too clever or too difficult to understand
  - Can obscure meaning and hamper understanding
  - Makes maintenance difficult

26 **DO NOT's**

- Create obscure side effects

27 **DO NOT's**

- Side effects (of a function call)
  - Modification of parameters
  - Modification of global variables
  - I/O operations

28 **DO NOT's**

- Obscure side effect - not obvious from casual code examination

29 **DO NOT's**

- Affect code understanding
  - E.g., global variable changed obscurely in a called module
  - E.g., some file I/O performed which is difficult to infer from function name and header information

30 **DO NOT's**

- Use identifier for multiple purposes
  - We often use same identifier to denote several temporary entities (e.g.., a temporary loop variable for computing and storing final result) for 'memory efficiency'

31 **DO NOT's**

- Variable should have descriptive name indicating purpose
  - Not possible if used for multiple purposes (affects readability and understanding)

32 **DO NOT's**

- Write 'lengthy' functions
  - Function length should not exceed 10 lines
  - Lengthy functions usually difficult to understand (likely to have different functions

coded together)
- Likely to have larger number of bugs

33 **DO NOT's**
- Use 'goto' (branching) statements indiscriminately
  - Makes a program unstructured and difficult to understand

34 **Reference**
- Rajib Mall (2018). Fundamentals of Software Engineering, 5th ed, PHI Learning Pvt Ltd. Chapter 10